Python Assignment 2: Data Structures - List, Dictionary, Set & Conditional Statements

List (Creation, Modification and Access)

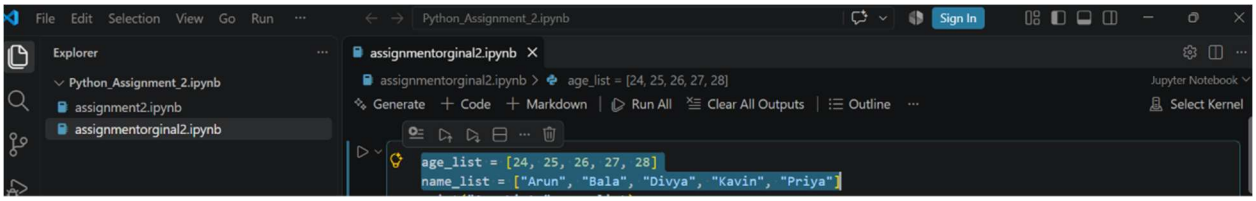
1. List Creation:

a. Create a list named age_list with five integer elements. For eg., [24, 25, 26, 27, 28]

b. Create a list named name list with five string elements.

Explanation: created a list called age list. A list is used to store multiple values in one variable. Here, the list contains five integer age values."

created another list called name_list, which contains five string values representing names



List Operations / Modifications:

Explanation :

used the append () method to add Yazhini to the end of the list

used insert () to add the value 30 at index 2. Python indexing starts from 0, so index 2 represents the third position.

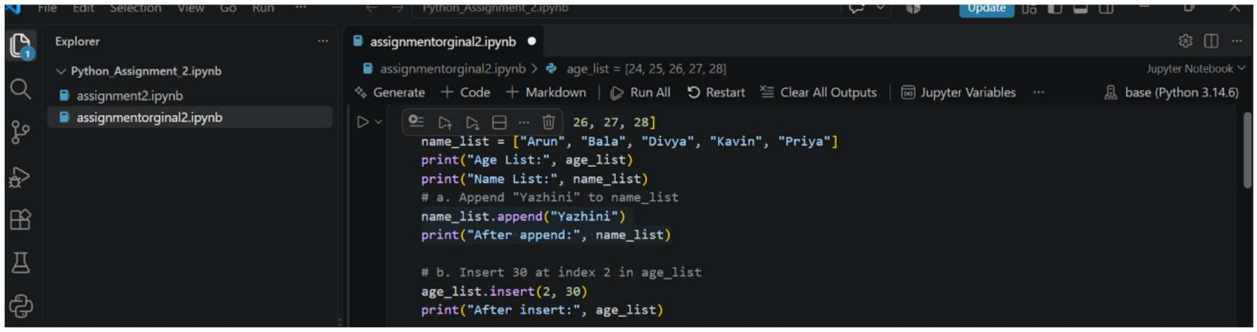
Used remove () method removes a specific value from the list. Here, I removed Yazhini

Used pop () removes the last element from the list when don't provide an index

Extend () is used to add multiple elements to an existing list. Here, I added 29, 30, and 26

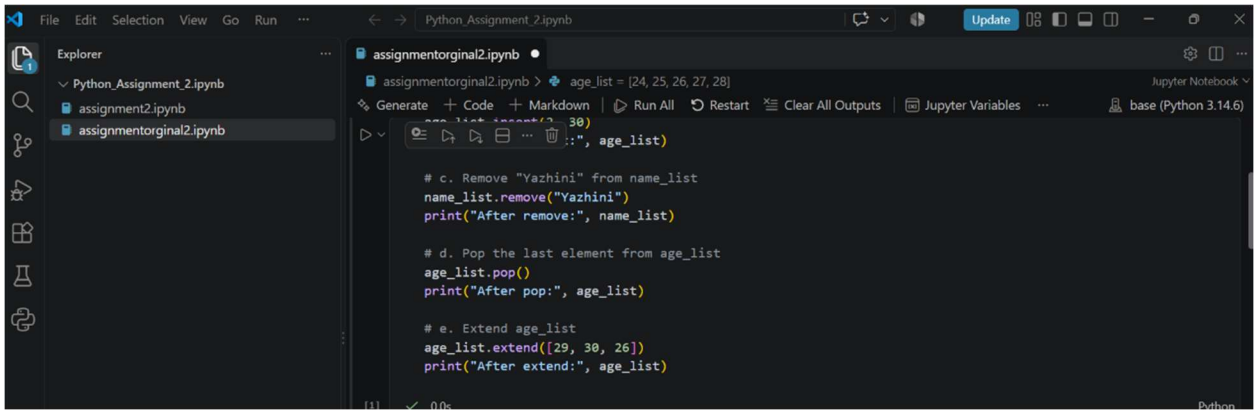
Used sort () arranges the elements. I used reverse=True to arrange the ages from highest to lowest

Max () finds the highest age, min() finds the lowest age, and sum() calculates the total of all ages



```
26, 27, 28]
name_list = ["Arun", "Bala", "Divya", "Kavin", "Priya"]
print("Age List:", age_list)
print("Name List:", name_list)
# a. Append "Yazhini" to name_list
name_list.append("Yazhini")
print("After append:", name_list)

# b. Insert 30 at index 2 in age_list
age_list.insert(2, 30)
print("After insert:", age_list)
```

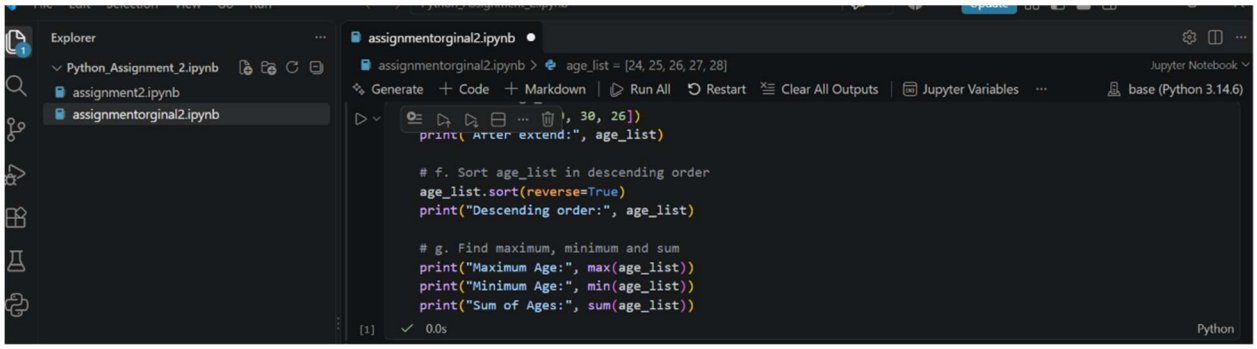


```
30)
::", age_list)

# c. Remove "Yazhini" from name_list
name_list.remove("Yazhini")
print("After remove:", name_list)

# d. Pop the last element from age_list
age_list.pop()
print("After pop:", age_list)

# e. Extend age_list
age_list.extend([29, 30, 26])
print("After extend:", age_list)
```

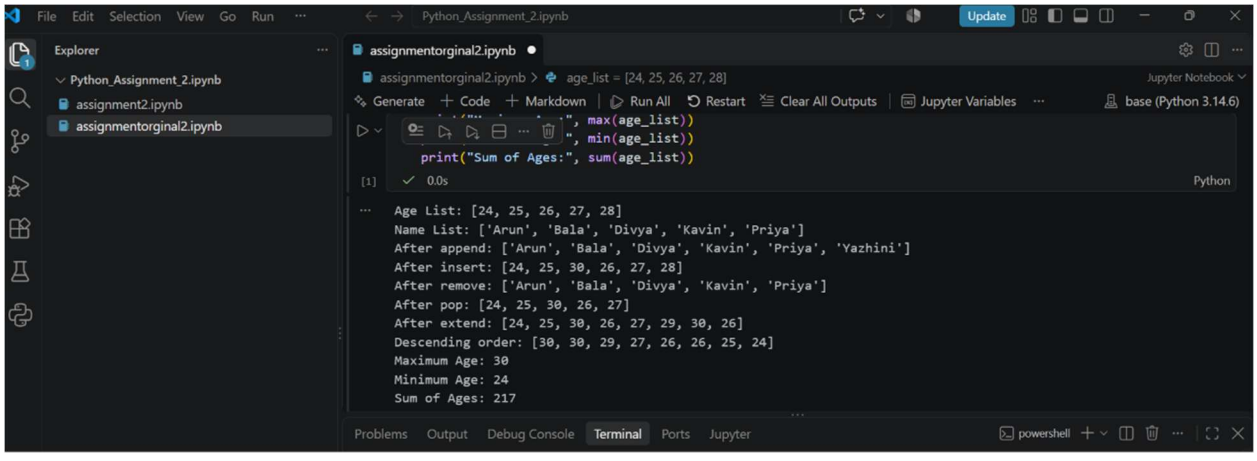


```
, 30, 26])
print("After extend:", age_list)

# f. Sort age_list in descending order
age_list.sort(reverse=True)
print("Descending order:", age_list)

# g. Find maximum, minimum and sum
print("Maximum Age:", max(age_list))
print("Minimum Age:", min(age_list))
print("Sum of Ages:", sum(age_list))
```

[1] ✓ 0.0s Python



```
", max(age_list))
", min(age_list))
print("Sum of Ages:", sum(age_list))
```

[1] ✓ 0.0s Python

```
Age List: [24, 25, 26, 27, 28]
Name List: ['Arun', 'Bala', 'Divya', 'Kavin', 'Priya']
After append: ['Arun', 'Bala', 'Divya', 'Kavin', 'Priya', 'Yazhini']
After insert: [24, 25, 30, 26, 27, 28]
After remove: ['Arun', 'Bala', 'Divya', 'Kavin', 'Priya']
After pop: [24, 25, 30, 26, 27]
After extend: [24, 25, 30, 26, 27, 29, 30, 26]
Descending order: [30, 30, 29, 27, 26, 26, 25, 24]
Maximum Age: 30
Minimum Age: 24
Sum of Ages: 217
```

Accessing List Elements

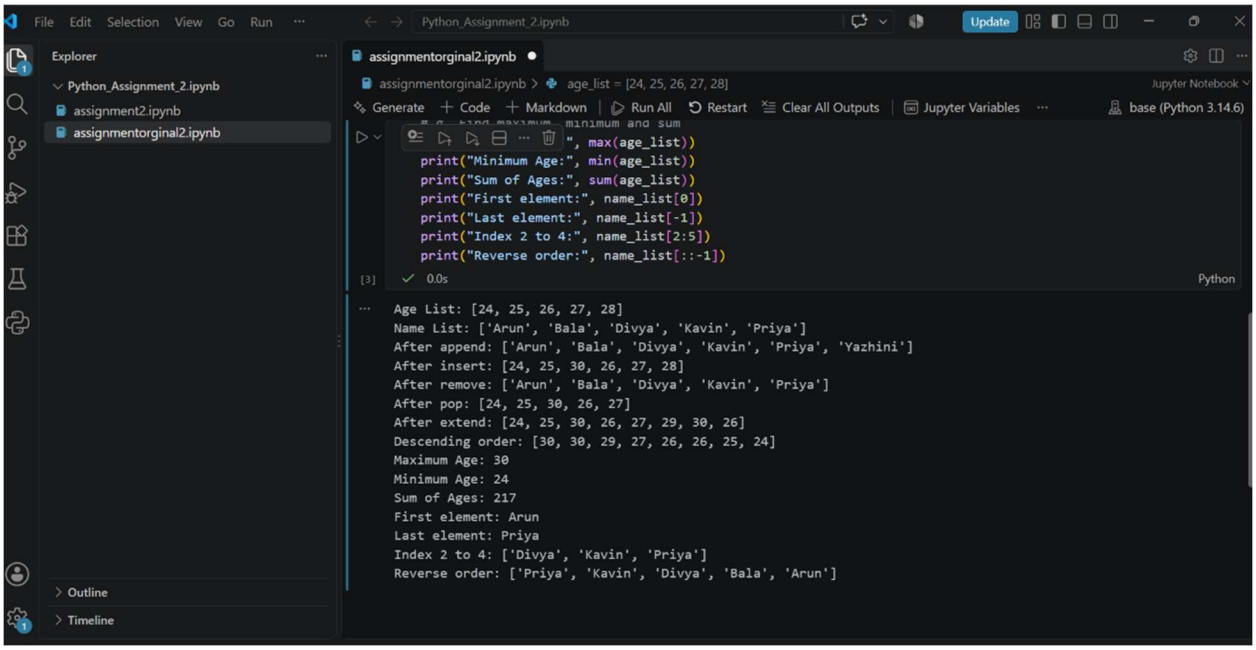
Explanation:

Index 0 gives the first element.

Index -1 gives the last element

list slicing. It starts from index 2 and stops before index 5, so it returns indexes 2, 3, and 4.

[::-1] reverses the list



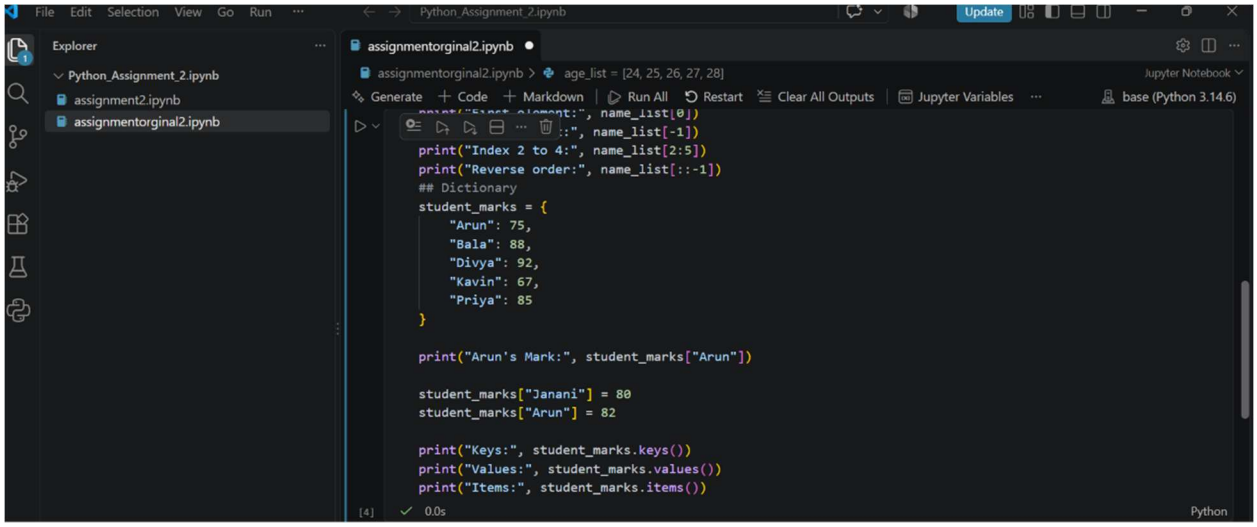
Dictionary (Creation, Modification and Access):

A dictionary stores information as key-value pairs. Here, the student's name is the key and the student's mark is the value

use the key Arun to retrieve Arun's mark

adds a new key Janani with the value 80

To update: student marks ["Arun"] = 82, Because Arun already exists, assigning 82 updates his previous mark



Sets (Operations):

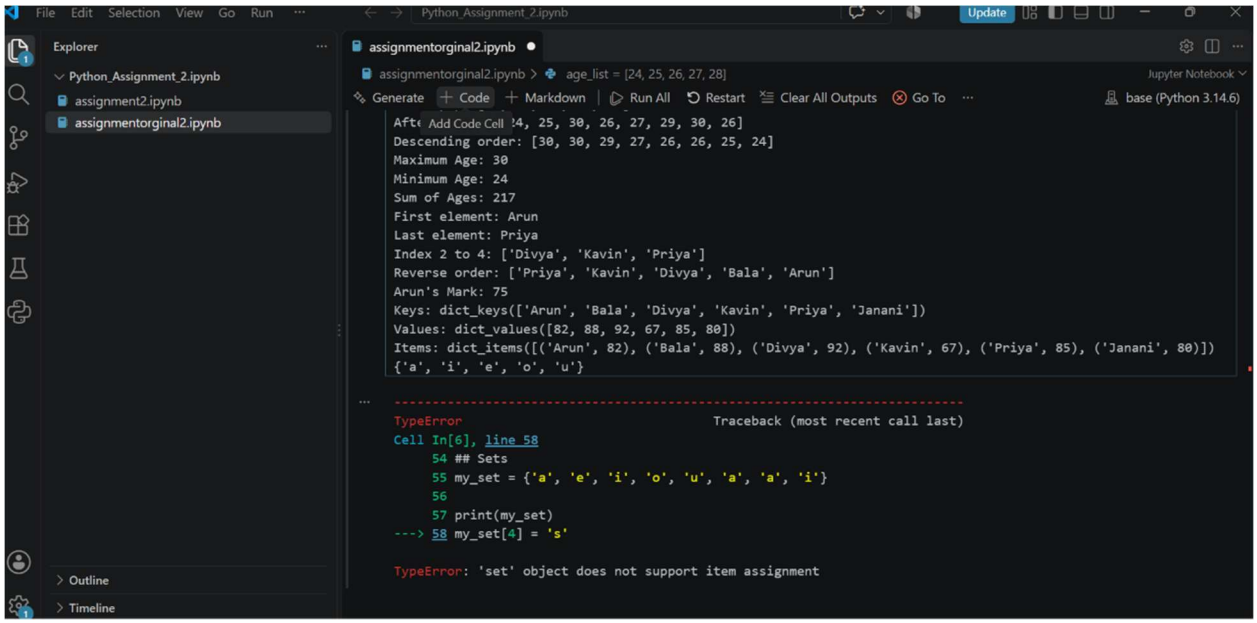
Explanation: A set stores only **unique values**. Therefore, repeated values such as a and i are automatically removed.

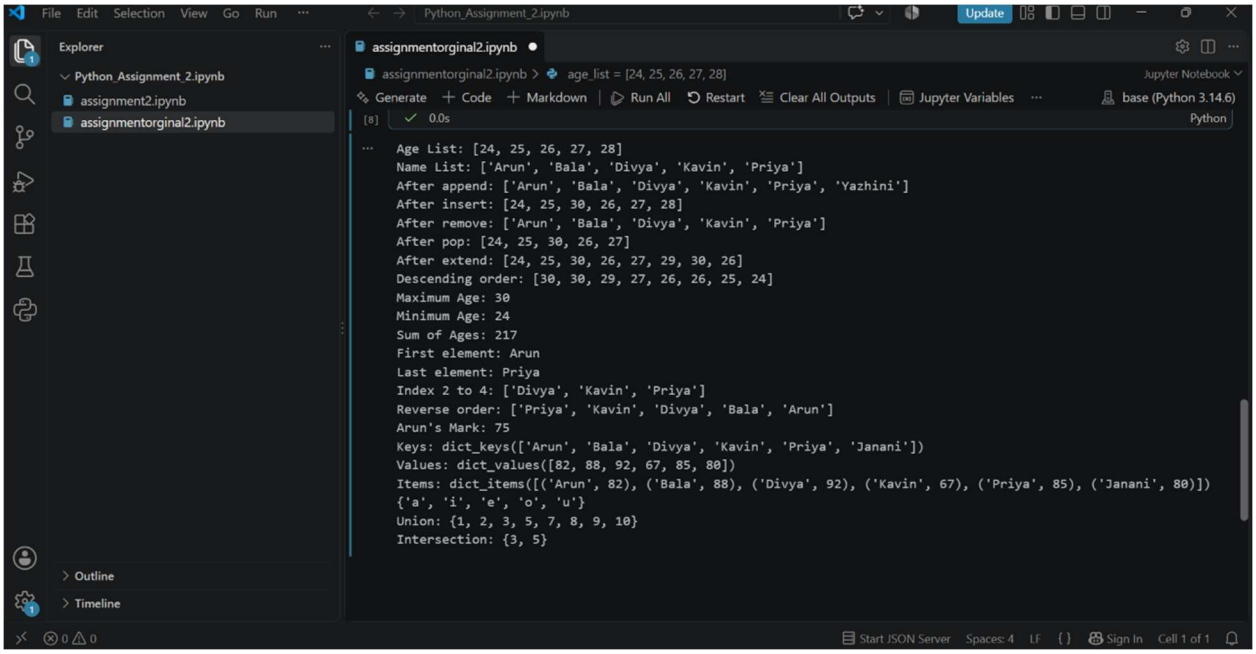
This produces an error because Python sets are unordered and don't support indexing. Therefore, we cannot access or modify a set element using my_set[4]

my_set[4] = 's'- TypeError: 'set' object does not support item assignment,Markdown cell that sets do not support indexing, so my_set[4] cannot be used.

Created two sets: set1 with values: {1, 3, 5, 7, 9} set2 with values: {2, 3, 5, 8, 10}

Union gives all unique elements from both sets, while intersection gives only elements that exist in both sets **my_set[4] = 's'- TypeError:**





Operators & Conditional Statements:

(IF, ELIF, ELSE)

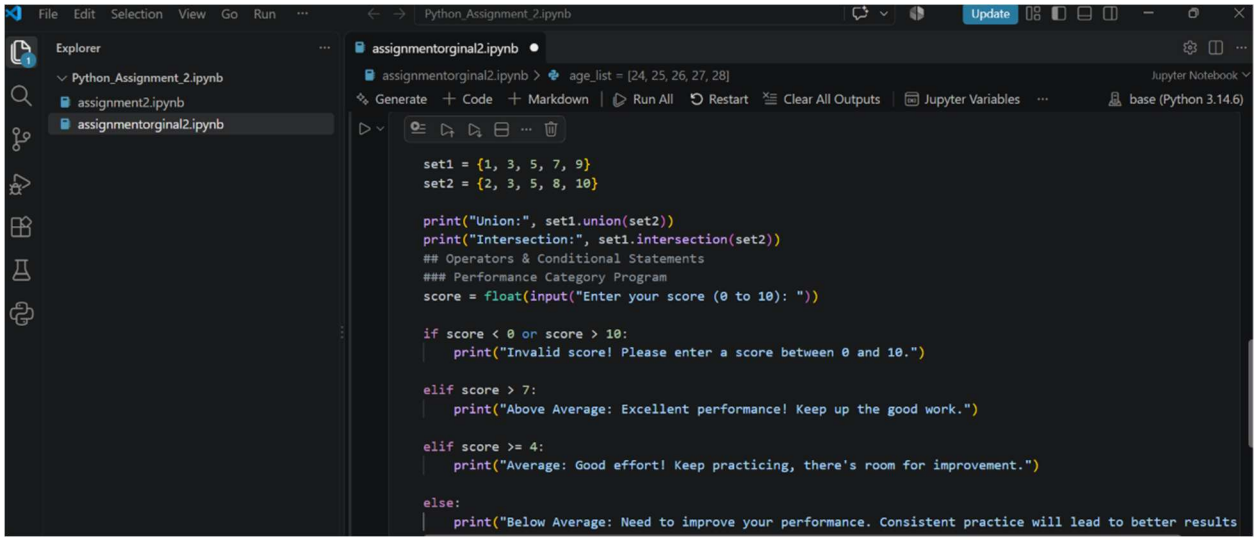
Performance Category Program:

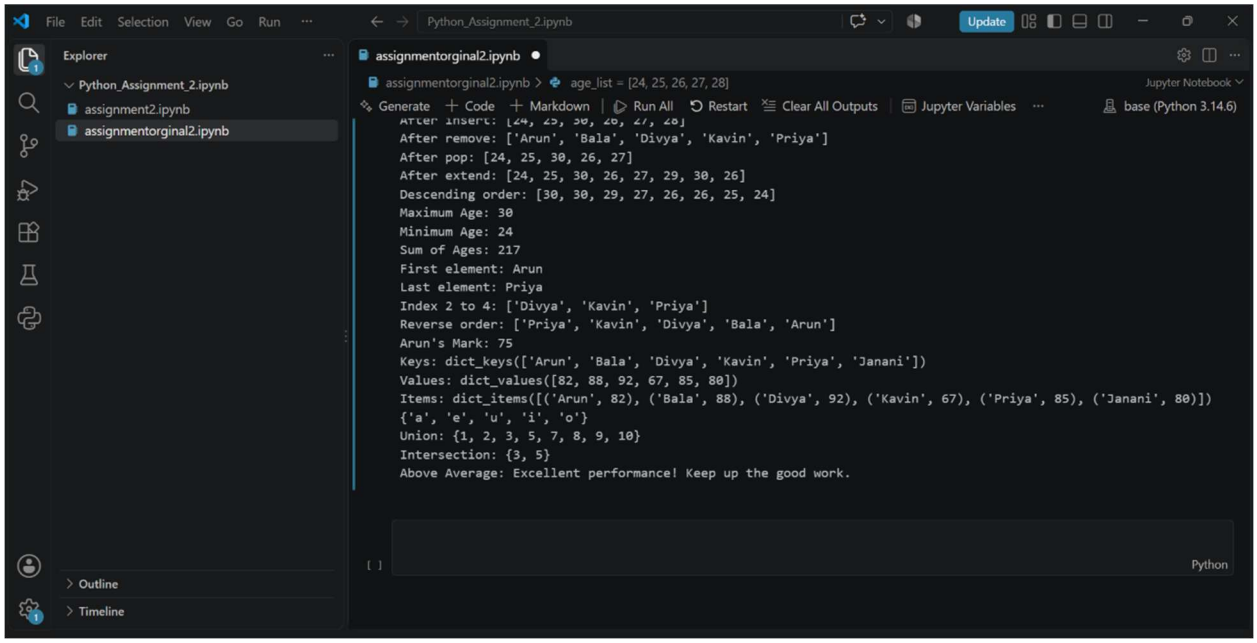
First, input() gets the score from the user and float() converts it into a number. Then I use if, elif, and else to make a decision based on the score. If the score is outside 0 to 10, it is invalid.

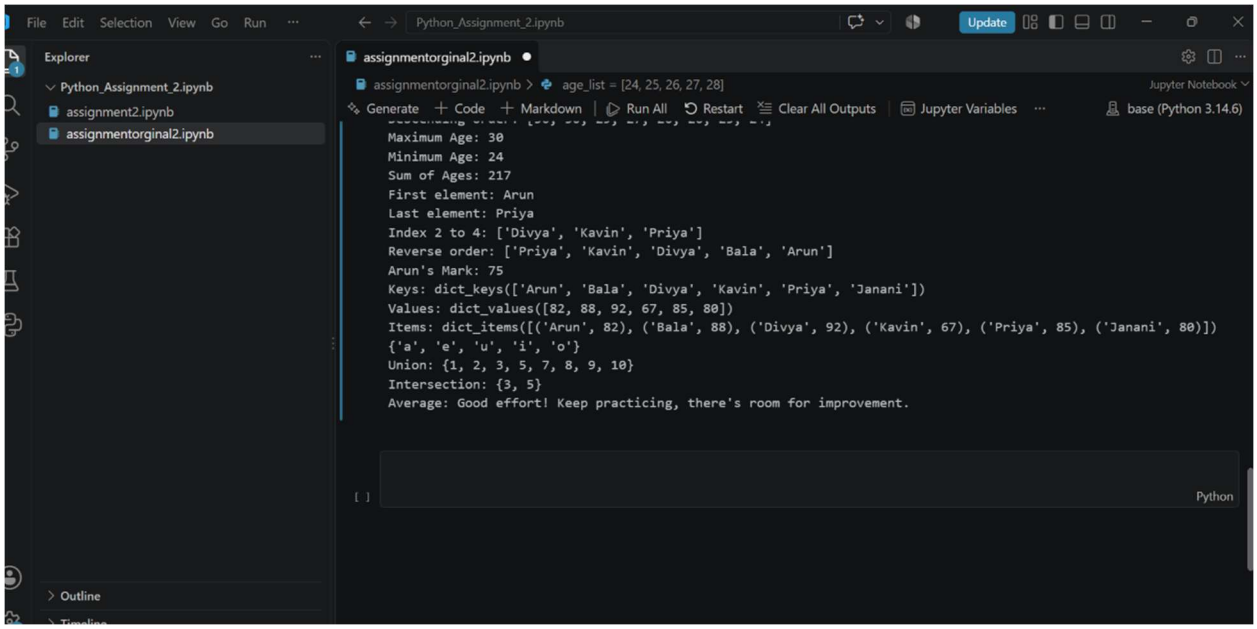
If it is greater than 7- Excellent performance! Keep up the good work, the performance is Above Average.

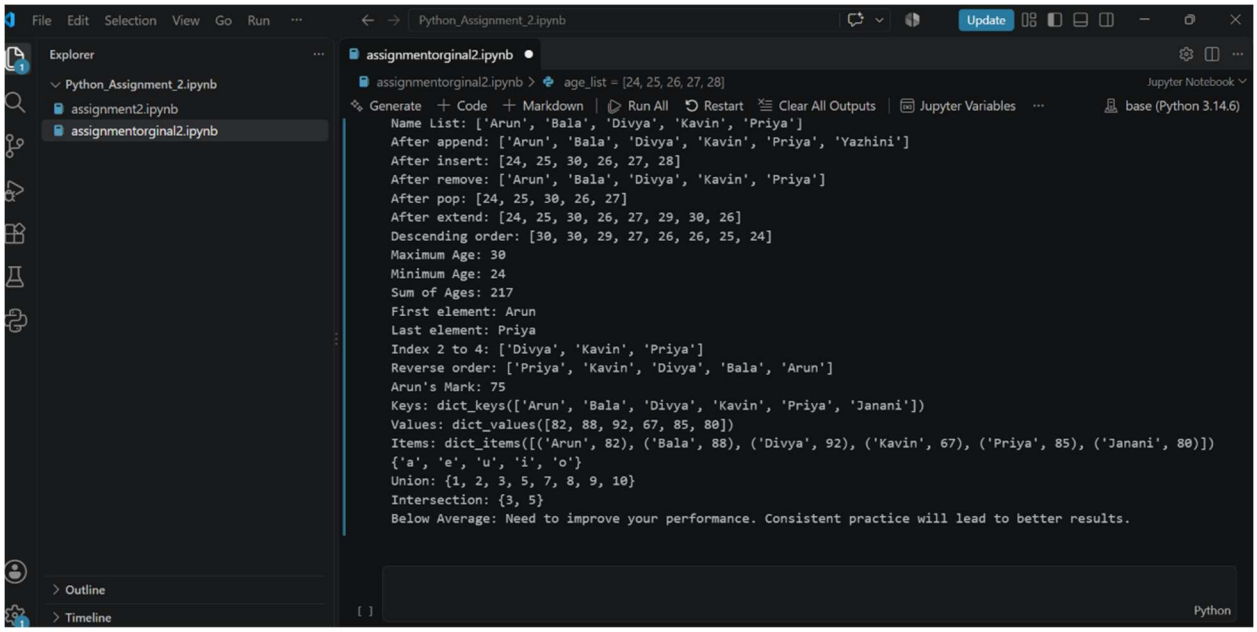
If it is between 4 and 7, it is Average -Good effort! Keep practicing, there's room for improvement

If it is between 0 and 3, it is Below Average- Need to improve your performance. Consistent practice will lead to better results


File Edit Selection View Go Run
Python_Assignment_2.ipynb
Update
Explorer
Python_Assignment_2.ipynb
assignment2.ipynb
assignmentorginal2.ipynb
assignmentorginal2.ipynb
assignmentorginal2.ipynb > age_list = [24, 25, 26, 27, 28]
Jupyter Notebook
Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables
base (Python 3.14.6)
set1 = {1, 3, 5, 7, 9}
set2 = {2, 3, 5, 8, 10}
print("Union:", set1.union(set2))
print("Intersection:", set1.intersection(set2))
Operators & Conditional Statements
Performance Category Program
score = float(input("Enter your score (0 to 10): "))
if score < 0 or score > 10:
print("Invalid score! Please enter a score between 0 and 10.")
elif score > 7:
print("Above Average: Excellent performance! Keep up the good work.")
elif score >= 4:
print("Average: Good effort! Keep practicing, there's room for improvement.")
else:
print("Below Average: Need to improve your performance. Consistent practice will lead to better results


File Edit Selection View Go Run
Python_Assignment_2.ipynb
Update
Explorer
Python_Assignment_2.ipynb
assignment2.ipynb
assignmentorginal2.ipynb
assignmentorginal2.ipynb
assignmentorginal2.ipynb > age_list = [24, 25, 26, 27, 28]
Jupyter Notebook
Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables
base (Python 3.14.6)
After insert: [24, 25, 30, 26, 27, 28]
After remove: ['Arun', 'Bala', 'Divya', 'Kavin', 'Priya']
After pop: [24, 25, 30, 26, 27]
After extend: [24, 25, 30, 26, 27, 29, 30, 26]
Descending order: [30, 30, 29, 27, 26, 26, 25, 24]
Maximum Age: 30
Minimum Age: 24
Sum of Ages: 217
First element: Arun
Last element: Priya
Index 2 to 4: ['Divya', 'Kavin', 'Priya']
Reverse order: ['Priya', 'Kavin', 'Divya', 'Bala', 'Arun']
Arun's Mark: 75
Keys: dict_keys(['Arun', 'Bala', 'Divya', 'Kavin', 'Priya', 'Janani'])
Values: dict_values([82, 88, 92, 67, 85, 80])
Items: dict_items([('Arun', 82), ('Bala', 88), ('Divya', 92), ('Kavin', 67), ('Priya', 85), ('Janani', 80)])
{'a', 'e', 'u', 'i', 'o'}
Union: {1, 2, 3, 5, 7, 8, 9, 10}
Intersection: {3, 5}
Above Average: Excellent performance! Keep up the good work.
[ ]
Python
Outline
Timeline


File Edit Selection View Go Run
Python_Assignment_2.ipynb
Update
Explorer
Python_Assignment_2.ipynb
assignment2.ipynb
assignmentorginal2.ipynb
assignmentorginal2.ipynb
assignmentorginal2.ipynb > age_list = [24, 25, 26, 27, 28]
Jupyter Notebook
Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables
base (Python 3.14.6)
Maximum Age: 30
Minimum Age: 24
Sum of Ages: 217
First element: Arun
Last element: Priya
Index 2 to 4: ['Divya', 'Kavin', 'Priya']
Reverse order: ['Priya', 'Kavin', 'Divya', 'Bala', 'Arun']
Arun's Mark: 75
Keys: dict_keys(['Arun', 'Bala', 'Divya', 'Kavin', 'Priya', 'Janani'])
Values: dict_values([82, 88, 92, 67, 85, 80])
Items: dict_items([('Arun', 82), ('Bala', 88), ('Divya', 92), ('Kavin', 67), ('Priya', 85), ('Janani', 80)])
{'a', 'e', 'u', 'i', 'o'}
Union: {1, 2, 3, 5, 7, 8, 9, 10}
Intersection: {3, 5}
Average: Good effort! Keep practicing, there's room for improvement.
[ ]
Python
Outline
Timeline


File Edit Selection View Go Run
Python_Assignment_2.ipynb
Update
Explorer
Python_Assignment_2.ipynb
assignment2.ipynb
assignmentorginal2.ipynb
assignmentorginal2.ipynb
assignmentorginal2.ipynb > age_list = [24, 25, 26, 27, 28]
Jupyter Notebook
Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables
base (Python 3.14.6)
Name List: ['Arun', 'Bala', 'Divya', 'Kavin', 'Priya']
After append: ['Arun', 'Bala', 'Divya', 'Kavin', 'Priya', 'Yazhini']
After insert: [24, 25, 30, 26, 27, 28]
After remove: ['Arun', 'Bala', 'Divya', 'Kavin', 'Priya']
After pop: [24, 25, 30, 26, 27]
After extend: [24, 25, 30, 26, 27, 29, 30, 26]
Descending order: [30, 30, 29, 27, 26, 26, 25, 24]
Maximum Age: 30
Minimum Age: 24
Sum of Ages: 217
First element: Arun
Last element: Priya
Index 2 to 4: ['Divya', 'Kavin', 'Priya']
Reverse order: ['Priya', 'Kavin', 'Divya', 'Bala', 'Arun']
Arun's Mark: 75
Keys: dict_keys(['Arun', 'Bala', 'Divya', 'Kavin', 'Priya', 'Janani'])
Values: dict_values([82, 88, 92, 67, 85, 80])
Items: dict_items([('Arun', 82), ('Bala', 88), ('Divya', 92), ('Kavin', 67), ('Priya', 85), ('Janani', 80)])
{'a', 'e', 'u', 'i', 'o'}
Union: {1, 2, 3, 5, 7, 8, 9, 10}
Intersection: {3, 5}
Below Average: Need to improve your performance. Consistent practice will lead to better results.
[ ]
Python
Outline
Timeline